Supplementary Material

The supplementary material is organized as follows. Section 1 presents the complete SQL definitions of all query chains used in our evaluation, including TPC-C1, TPC-C2, S-C1, S-C2, OR-C1, and OR-C2. For each chain, we provide the full sequence of queries, a brief description of its analytical intent, and the variants used in our experiments. Section 2 visualizes the data flow and query dependencies of each chain using flowcharts, highlighting the branching and merging structures.

1 QUERY CHAIN DEFINITIONS

1.1 TPC-H Dataset Chains

1.1.1 Chain TPC-C1: Supplier Performance Analysis. This chain models a supplier performance analysis. It starts from computing the max-min discount range for each supplier-part pair (Q1-Q3). Then it ranks parts by discount range within each supplier and selects the top-2 discounted parts (Q4-Q5). In parallel, it computes return flag statistics—total quantity grouped by return flag (Q7-Q8) and overall (Q9)—and calculates the return ratio (return/total) for 'R' vs. non-'R' flags (Q10-Q12). Finally, it merges these branches by multiplying the top-part count with the return ratio for each supplier (Q13-Q14).

```
--  Input Query: TPC-C1.Q1
Q1: SELECT l_suppkey,l_partkey,MAX(l_discount) AS m0 FROM lineitem GROUP BY l_suppkey,l_partkey ORDER BY l_suppkey,l_partkey

--  Input Query: TPC-C1.Q2
Q2: SELECT l_suppkey,l_partkey,MIN(l_discount) AS m0 FROM lineitem GROUP BY l_suppkey,l_partkey ORDER BY l_suppkey,l_partkey

Q3: SELECT Q1.l_suppkey,Q1.l_partkey,Q1.m0 - Q2.m0 AS m0 FROM Q1, Q2 WHERE Q1.l_suppkey=Q2.l_suppkey AND Q1.l_partkey=Q2.l_partkey
    ORDER BY Q1.l_suppkey,Q1.l_partkey

Q4: SELECT l_suppkey,l_partkey,Rank() OVER(PARTITION BY l_partkey ORDER BY m0 DESC) AS m0 FROM Q3

Q5: SELECT l_suppkey,l_partkey,m0 AS m0 FROM Q4 WHERE m0<3

Q6: SELECT l_suppkey,COUNT(*) AS m0 FROM Q5 GROUP BY l_suppkey ORDER BY l_suppkey

--  Input Query: TPC-C1.Q7
Q7: SELECT l_suppkey,l_returnflag,l_linestatus,l_partkey,SUM(l_quantity) AS m0 FROM lineitem
    GROUP BY l_suppkey,l_returnflag,l_linestatus,l_partkey ORDER BY l_suppkey,l_returnflag,l_linestatus,l_partkey

Q8: SELECT l_suppkey,l_returnflag,SUM(m0) AS m0 FROM Q7 GROUP BY l_suppkey,l_returnflag ORDER BY l_suppkey,l_returnflag

Q9: SELECT l_suppkey,SUM(m0) AS m0 FROM Q7 GROUP BY l_suppkey ORDER BY l_suppkey

Q10: SELECT Q8.l_suppkey,Q8.l_returnflag,Q8.m0 / Q9.m0 AS m0 FROM Q8, Q9 WHERE Q8.l_suppkey=Q9.l_suppkey ORDER BY Q8.l_suppkey,Q8.l_returnflag

Q11: SELECT l_suppkey,m0 AS m0 FROM Q10 WHERE l_returnflag='R'

Q12: SELECT l_suppkey,AVG(m0) AS m0 FROM Q10 WHERE l_returnflag!='R' GROUP BY l_suppkey ORDER BY l_suppkey

Q13: SELECT Q11.l_suppkey,Q11.m0 / Q12.m0 AS m0 FROM Q11, Q12 WHERE Q11.l_suppkey=Q12.l_suppkey ORDER BY Q11.l_suppkey

Q14: SELECT Q6.l_suppkey,Q6.m0 * Q13.m0 AS m0 FROM Q6, Q13 WHERE Q6.l_suppkey=Q13.l_suppkey ORDER BY Q6.l_suppkey
```

1.1.2 Chain TPC-C2: Monthly Shipment Trend Analysis. This chain models a month-over-month trend analysis for supplier-part shipments. It starts from aggregating daily shipment quantities for each part (Q1). Then it truncates dates to month and re-aggregates to monthly level (Q2-Q3). Next, it computes each part's monthly share of total monthly quantity (Q4-Q5) and converts it to percentage (Q6). Finally, it calculates the month-over-month growth in percentage share by comparing each month with the next month (Q7-Q8) and filters for positive growth (Q9)

```
-- Input Query: TPC-C2.Q1
Q1: SELECT l_shipdate,l_partkey,SUM(l_quantity) AS m0 FROM lineitem GROUP BY l_shipdate,l_partkey ORDER BY l_shipdate,l_partkey

Q2: SELECT DATE_TRUNC('month',l_shipdate) AS l_shipdate,l_partkey,m0 AS m0 FROM Q1

Q3: SELECT l_shipdate,l_partkey,SUM(m0) AS m0 FROM Q2 GROUP BY l_shipdate,l_partkey ORDER BY l_shipdate,l_partkey

Q4: SELECT l_shipdate,SUM(m0) AS m0 FROM Q3 GROUP BY l_shipdate ORDER BY l_shipdate

Q5: SELECT Q3.l_shipdate,Q3.l_partkey,Q3.m0 / Q4.m0 AS m0 FROM Q3, Q4 WHERE Q3.l_shipdate=Q4.l_shipdate ORDER BY Q3.l_shipdate,Q3.l_partkey
```

```
Q6: SELECT l_shipdate,l_partkey,m0 * 100 AS m0 FROM Q5

Q7: SELECT DATE_ADD(l_shipdate,1 month) AS l_shipdate,l_partkey,m0 AS m0 FROM Q6

Q8: SELECT Q6.l_shipdate,Q6.l_partkey,Q6.m0 - Q7.m0 AS m0 FROM Q6, Q7 WHERE Q6.l_shipdate=Q7.l_shipdate AND Q6.l_partkey=Q7.l_partkey
    ORDER BY Q6.l_shipdate,Q6.l_partkey

Q9: SELECT l_shipdate,l_partkey,m0 AS m0 FROM Q8 WHERE m0>0
```

1.1.3 Chain TPC-C2-V1 (Dimension Addition). This variant extends each query (except the aggregation Q4) with an additional dimension l_suppkey. For example, Q1 becomes:

```
Q1: SELECT l_shipdate, l_partkey, l_suppkey, SUM(l_quantity) AS m0 FROM lineitem GROUP BY l_shipdate, l_partkey, l_suppkey
    ORDER BY l_shipdate, l_partkey, l_suppkey
```

All subsequent queries in the chain are modified analogously to propagate the new dimension through joins and aggregations, while Q4 remains unchanged as it aggregates to the monthly level.

1.1.4 Chain TPC-C2-V2 (Dimension Projection). Starting from TPC-C2-V1, this variant projects out the l_suppkey dimension in queries Q6 through Q9, simulating generalized cube computation during query chain execution. After projection, a single dimension combination (l_shipdate, l_partkey) can correspond to multiple cells—one for each distinct l_suppkey value from the earlier stage—demonstrating the model's ability to handle non-standard cube structures where coordinates and measures have one-to-many mappings.

1.1.5 Chain TPC-C2-V3 (Dimension Reordering). Based on TPC-C2-V1, this variant reorders the dimension sequence in all queries (except Q4) from (l_shipdate, l_partkey, l_suppkey) to (l_partkey, l_suppkey, l_shipdate). Unlike TPC-C2-V1 (including the original TPC-C2), this reordering allows Q2 to preserve the input coordinate order after Date_Trunc. Under pipelined execution, this reordering also causes Q4 to transition from partially blocking to fully blocking.

1.2 Stocks Dataset Chains

1.2.1 Chain S-C1: Stock Volatility Anomaly Detection. This chain analyzes high-frequency stock trading data to detect intraday volatility anomalies and identify stocks with unusually active abnormal movements. It starts by computing the 2-minute rolling max-min range (Q1-Q3) and its ratio to the rolling min (Q4), converting to percentage (Q5). It then detects outliers by subtracting the daily 75th percentile (Q6) and filtering positive deviations (Q7-Q8). Finally, it counts anomalous minutes per stock per day (Q9), identifies days with abnormally high counts via another 75th percentile filter (Q10-Q11), and outputs stocks with persistently high anomaly frequency (Q12).

```
-- Input Query: S-C1.Q1
Q1: SELECT Date,Code,Time,Max(High) OVER(ORDER BY Date,Code,Time ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS m0 FROM Stock ORDER BY Date,Code,Time

-- Input Query: S-C1.Q2
Q2: SELECT Date,Code,Time,Min(Low) OVER(ORDER BY Date,Code,Time ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS m0 FROM Stock ORDER BY Date,Code,Time

Q3: SELECT Q1.Date,Q1.Code,Q1.Time,Q1.m0 - Q2.m0 AS m0 FROM Q1, Q2 WHERE Q1.Date=Q2.Date AND Q1.Code=Q2.Code AND Q1.Time=Q2.Time
    ORDER BY Q1.Date,Q1.Code,Q1.Time

Q4: SELECT Q3.Date,Q3.Code,Q3.Time,Q3.m0 / Q2.m0 AS m0 FROM Q3, Q2 WHERE Q3.Date=Q2.Date AND Q3.Code=Q2.Code AND Q3.Time=Q2.Time
    ORDER BY Q3.Date,Q3.Code,Q3.Time

Q5: SELECT Date,Code,Time,m0 * 100 AS m0 FROM Q4 WHERE m0!=0

Q6: SELECT Date,Quantile(m0, 0.75) AS m0 FROM Q5 GROUP BY Date ORDER BY Date

Q7: SELECT Q5.Date,Q5.Code,Q5.Time,Q5.m0 - Q6.m0 AS m0 FROM Q5, Q6 WHERE Q5.Date=Q6.Date ORDER BY Q5.Date,Q5.Code,Q5.Time

Q8: SELECT Date,Code,Time,m0 AS m0 FROM Q7 WHERE m0>0

Q9: SELECT Date,Code,COUNT(*) AS m0 FROM Q8 GROUP BY Date,Code ORDER BY Date,Code

Q10: SELECT Date,Quantile(m0, 0.75) AS m0 FROM Q9 GROUP BY Date ORDER BY Date

Q11: SELECT Q9.Date,Q9.Code,Q9.m0 - Q10.m0 AS m0 FROM Q9, Q10 WHERE Q9.Date=Q10.Date ORDER BY Q9.Date,Q9.Code

Q12: SELECT Date,Code,m0 AS m0 FROM Q11 WHERE m0>0
```

1.2.2 Chain S-C2: Stock Pearson Correlation. This chain computes the Pearson correlation coefficient between each stock's minute-over-minute returns and a reference stock (Code 300083 on 2025-02-26). It starts by calculating minute-over-minute price differences (Q1-Q3) and their deviations from the daily mean (Q4-Q5). It then isolates the reference stock's return series (Q6) and computes the dot product between each stock's returns and the reference (Q7-Q8), as well as each stock's squared returns for L2 norm (Q9-Q11). Finally, it calculates the reference's L2 norm (Q12-Q13) and computes the correlation coefficient as covariance divided by the product of norms (Q14).

```
--  Input Query: S-C2.Q1
Q1: SELECT Date,Code,Time,SUM(Price) AS m0 FROM Stock GROUP BY Date,Code,Time ORDER BY Date,Code,Time

Q2: SELECT Date,Code,Time + INTERVAL 1 MINUTE AS Time,m0 AS m0 FROM Q1

Q3: SELECT Q1.Date,Q1.Code,Q1.Time,Q1.m0 - Q2.m0 AS m0 FROM Q1, Q2 WHERE Q1.Date=Q2.Date AND Q1.Code=Q2.Code AND Q1.Time=Q2.Time
    ORDER BY Q1.Date,Q1.Code,Q1.Time

Q4: SELECT Date,Code,AVG(m0) AS m0 FROM Q3 GROUP BY Date,Code ORDER BY Date,Code

Q5: SELECT Q3.Date,Q3.Code,Q3.Time,Q3.m0 - Q4.m0 AS m0 FROM Q3, Q4 WHERE Q3.Date=Q4.Date AND Q3.Code=Q4.Code ORDER BY Q3.Date,Q3.Code,Q3.Time

Q6: SELECT Time,m0 AS m0 FROM Q5 WHERE Date='2025-02-26' AND Code='300083'

Q7: SELECT Q5.Date,Q5.Code,Q5.Time,Q5.m0 * Q6.m0 AS m0 FROM Q5, Q6 WHERE Q5.Time=Q6.Time ORDER BY Q5.Date,Q5.Code,Q5.Time

Q8: SELECT Date,Code,AVG(m0) AS m0 FROM Q7 GROUP BY Date,Code ORDER BY Date,Code

Q9: SELECT Date,Code,Time,Pow(m0,2) AS m0 FROM Q5

Q10: SELECT Date,Code,AVG(m0) AS m0 FROM Q9 GROUP BY Date,Code ORDER BY Date,Code

Q11: SELECT Date,Code,Pow(m0,0.5) AS m0 FROM Q10 WHERE m0!=0

Q12: SELECT m0 AS m0 FROM Q11 WHERE Date='2025-02-26' AND Code='300083'

Q13: SELECT Q11.Date,Q11.Code,Q11.m0 * Q12.m0 AS m0 FROM Q11, Q12 ORDER BY Q11.Date,Q11.Code

Q14: SELECT Q8.Date,Q8.Code,Q8.m0 / Q13.m0 AS m0 FROM Q8, Q13 WHERE Q8.Date=Q13.Date AND Q8.Code=Q13.Code ORDER BY Q8.Date,Q8.Code
```

1.2.3 Chain S-C1-V2 (Dimension Projection). Based on the original S-C1, this variant projects out the Time dimension in queries Q5, Q7, and Q8, while keeping the rest of the chain unchanged.

1.2.4 Chain S-C1-V3 (Partial Dimension Reordering). Based on the original S-C1, this variant reorders the dimension sequence in queries Q1-Q5 and Q7-Q8 from (Date, Code, Time) to (Code, Date, Time), while keeping Q6 and the tail of the chain (Q9-Q12) unchanged. This partial reordering affects the intermediate cube layout during the early stage of anomaly detection, but preserves the original dimension order for final outputs.

1.2.5 Chain S-C2-V4 (Measure Addition). Based on the original S-C2, this variant adds a Volume measure (m1) to every query in the chain. For example, Q1 becomes:

```
Q1: SELECT Date, Code, Time, SUM(Price) AS m0, SUM(Volume) AS m1 FROM Stock GROUP BY Date, Code, Time ORDER BY Date, Code, Time
```

All subsequent queries are modified analogously to propagate and compute both measures throughout the Pearson correlation computation, demonstrating generalized cube behavior where multiple measures coexist and evolve along the query chain.

1.3 Online Retail Dataset Chains

1.3.1 Chain OR-C1: Market Basket Analysis (Lift). This chain computes the lift measure for item pairs to identify association rules. It starts by generating all item pairs co-occurring in the same invoice (Q1-Q3). It then counts the co-occurrence frequency for each pair (Q4) and divides by the total number of invoices to get confidence (Q5-Q6). Next, it calculates each item's individual frequency (Q7-Q8) to derive support (Q9). Finally, it computes lift as confidence divided by support for each item pair (Q10).

```
--  Input Query: OR-C1.Q1
Q1: SELECT Invoice,StockCode FROM retail GROUP BY Invoice,StockCode ORDER BY Invoice,StockCode

--  Input Query: OR-C1.Q2
Q2: SELECT Invoice,StockCode FROM retail GROUP BY Invoice,StockCode ORDER BY Invoice,StockCode

Q3: SELECT Q1.Invoice,Q1.StockCode,Q2.StockCode AS StockCode2 FROM Q1, Q2
    WHERE Q1.Invoice=Q2.Invoice AND Q1.StockCode<Q2.StockCode ORDER BY Q1.Invoice,Q1.StockCode,Q2.StockCode

Q4: SELECT StockCode,StockCode2,COUNT(*) AS m0 FROM Q3 GROUP BY StockCode,StockCode2 ORDER BY StockCode,StockCode2

Q5: SELECT COUNT(DISTINCT Invoice) AS m0 FROM Q3

Q6: SELECT Q4.StockCode,Q4.StockCode2,Q4.m0 / Q5.m0 AS m0 FROM Q4, Q5 ORDER BY Q4.StockCode,Q4.StockCode2

--  Input Query: OR-C1.Q7
Q7: SELECT StockCode,COUNT(DISTINCT Invoice) AS m0 FROM retail GROUP BY StockCode ORDER BY StockCode

--  Input Query: OR-C1.Q8
```

```
Q8: SELECT COUNT(DISTINCT Invoice) AS m0 FROM retail

Q9: SELECT Q7.StockCode,Q7.m0 / Q8.m0 AS m0 FROM Q7, Q8 ORDER BY Q7.StockCode

Q10: SELECT Q6.StockCode,Q6.StockCode2,Q6.m0 / Q9.m0 AS m0 FROM Q6, Q9 WHERE Q6.StockCode=Q9.StockCode ORDER BY Q6.StockCode,Q6.StockCode2
```

1.3.2 Chain OR-C2: Item-to-Item Cosine Similarity. This chain computes cosine similarity between items based on customer purchase patterns. It starts by building a customer-item matrix where each cell represents the total spend (Q1-Q2). It then computes the dot product between each pair of item vectors (Q3-Q4). In parallel, it calculates the L2 norm for each item by squaring, summing, and square-rooting the customer spends (Q5-Q7). Finally, it normalizes the dot product by the product of the two items' norms to obtain cosine similarity (Q8-Q9).

```
--  Input Query: OR-C2.Q1
Q1: SELECT CustomerID,StockCode,SUM(Quantity*Price) AS m0 FROM retail WHERE CustomerID IS NOT NULL
    GROUP BY CustomerID,StockCode ORDER BY CustomerID,StockCode

--  Input Query: OR-C2.Q2
Q2: SELECT CustomerID,StockCode,SUM(Quantity*Price) AS m0 FROM retail WHERE CustomerID IS NOT NULL
    GROUP BY CustomerID,StockCode ORDER BY CustomerID,StockCode

Q3: SELECT Q1.CustomerID,Q1.StockCode,Q2.StockCode AS StockCode2,Q1.m0 * Q2.m0 AS m0 FROM Q1, Q2 WHERE Q1.CustomerID=Q2.CustomerID
    ORDER BY Q1.CustomerID,Q1.StockCode,Q2.StockCode

Q4: SELECT StockCode,StockCode2,SUM(m0) AS m0 FROM Q3 GROUP BY StockCode,StockCode2 ORDER BY StockCode,StockCode2

Q5: SELECT CustomerID,StockCode,Pow(m0,2) AS m0 FROM Q1

Q6: SELECT StockCode,SUM(m0) AS m0 FROM Q5 GROUP BY StockCode ORDER BY StockCode

Q7: SELECT StockCode,Pow(m0,0.5) AS m0 FROM Q6

Q8: SELECT Q4.StockCode,Q4.StockCode2,Q4.m0 / Q7.m0 AS m0 FROM Q4, Q7 WHERE Q4.StockCode=Q7.StockCode ORDER BY Q4.StockCode,Q4.StockCode2

Q9: SELECT Q8.StockCode,Q8.StockCode2,Q8.m0 / Q7.m0 AS m0 FROM Q8, Q7 WHERE Q8.StockCode2=Q7.StockCode ORDER BY Q8.StockCode,Q8.StockCode2
```

1.3.3 Chain OR-C1-V3 (Dimension Reordering). Based on the original OR-C1, this variant reorders the dimension sequence in Q1 and Q2 from (Invoice, StockCode) to (StockCode, Invoice), and in Q3 from (Invoice, StockCode, StockCode2) to (StockCode, StockCode2, Invoice), while keeping the rest of the chain unchanged. This reordering alters the data access pattern from invoice-first to item-first, affecting how intermediate cubes are laid out and traversed during market basket analysis.

2 QUERY CHAIN FLOWCHARTS

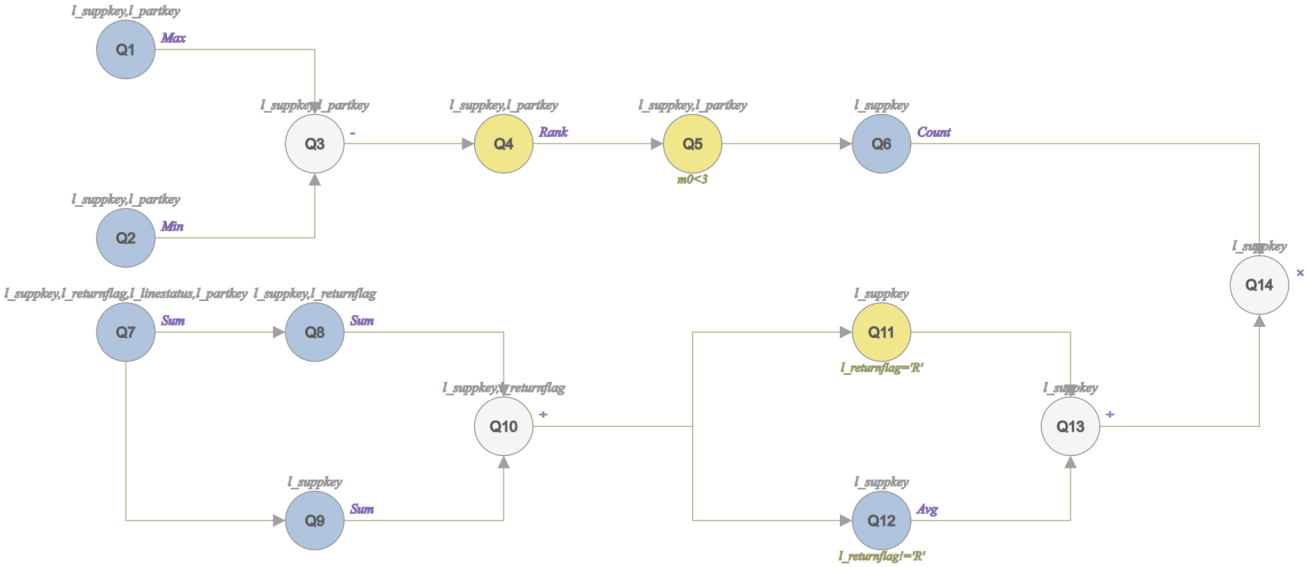

Figure 1: TPC-C1

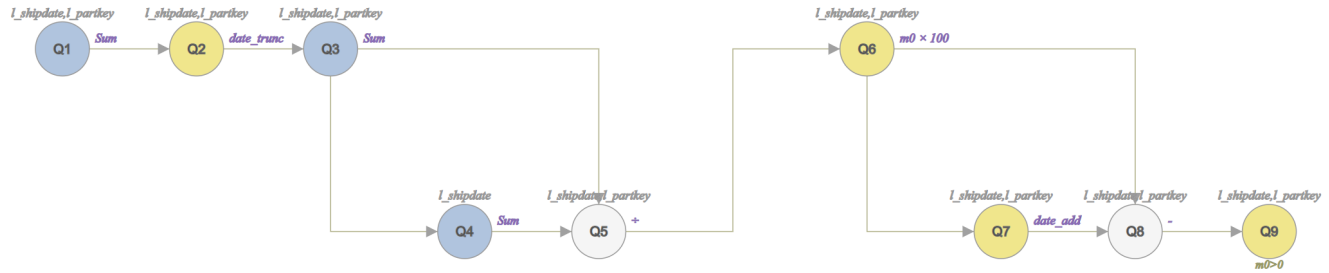

Figure 2: TPC-C2

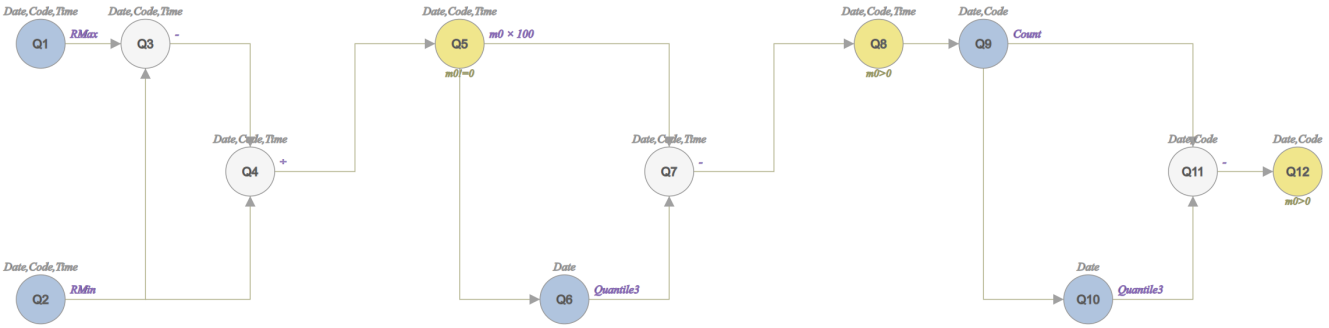

Figure 3: S-C1

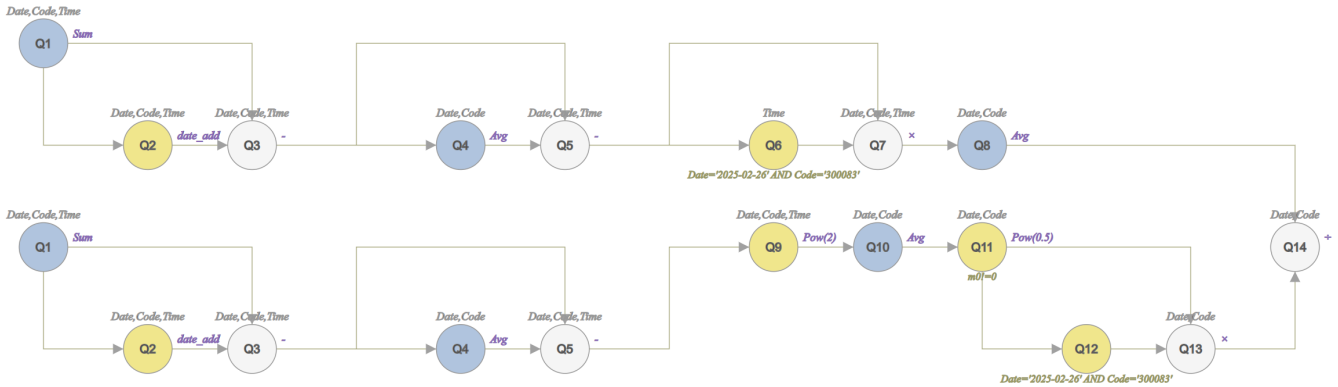

Figure 4: S-C2

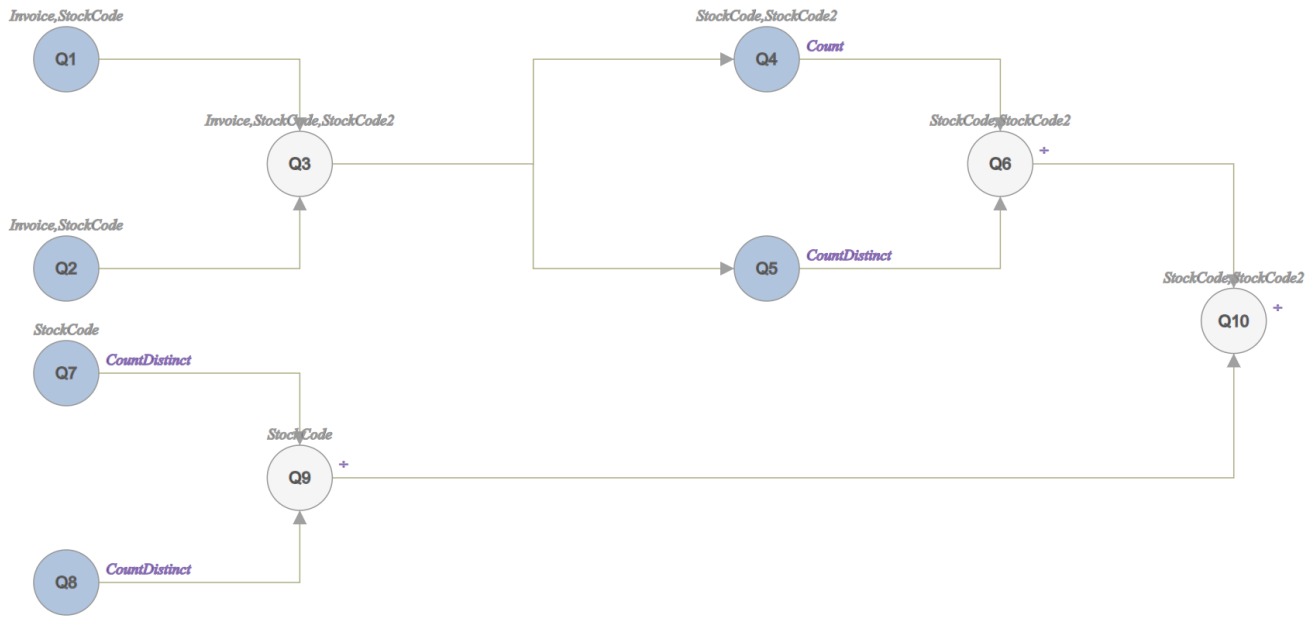

Figure 5: OR-C1

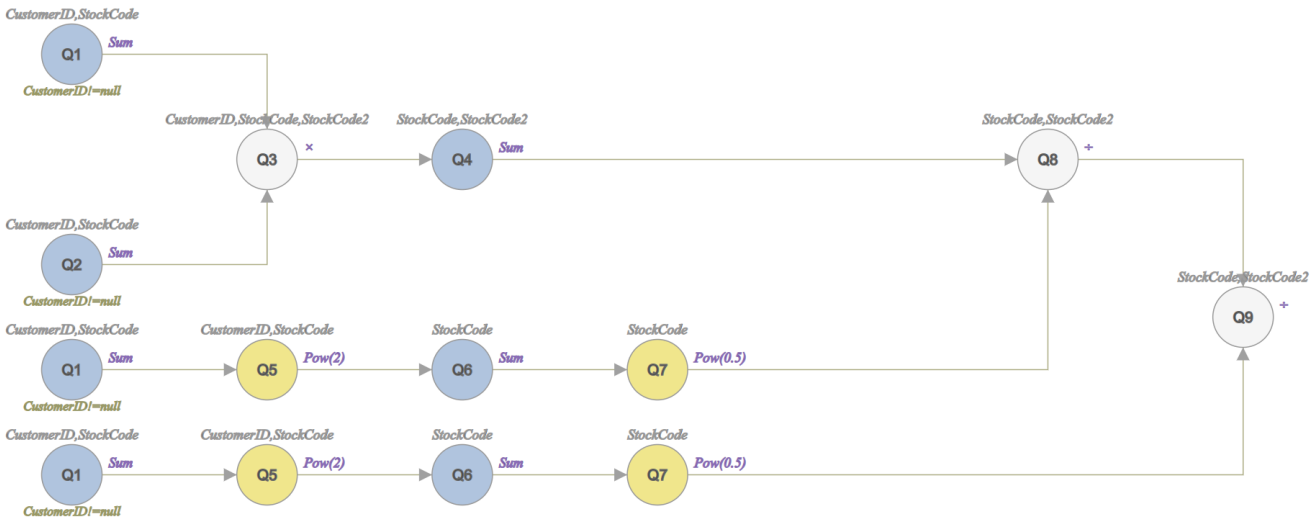

Figure 6: OR-C2